CHƯƠNG 4 · BÀI 3

4.3. Nhu cầu đồng bộ hóa (synchronisation)

Nội dung trích từ Bài giảng Hệ điều hành – Nguyễn Thị Hữu Phương, Bộ môn CNPM, Khoa CNTT.

Mục tiêu bài học

- Hiểu hai yêu cầu đồng bộ: độc quyền truy xuất và phối hợp.
- Nắm được khái niệm tranh đoạt điều khiển (race condition) và miền găng (critical section).
- Biết các điều kiện một giải pháp đồng bộ tốt phải thỏa mãn.

Trong một hệ thống cho phép các tiến trình liên lạc với nhau, bao giờ hệ điều hành cũng cần cung cấp kèm theo những cơ chế đồng bộ hóa để bảo đảm hoạt động của các tiến trình đồng hành không tác động sai lệch đến nhau vì các lý do sau đây:

4.3.1 Yêu cầu độc quyền truy xuất (Mutual exclusion)

Các tài nguyên trong hệ thống được phân thành hai loại: tài nguyên có thể chia sẻ cho phép nhiều tiến trình đồng thời truy xuất, và tài nguyên không thể chia sẻ chỉ chấp nhận một (hay một số lượng hạn chế) tiến trình sử dụng tại một thời điểm. Tính không thể chia sẻ của tài nguyên thường có nguồn gốc từ một trong hai nguyên nhân sau đây:

– Đặc tính cấu tạo phần cứng của tài nguyên không cho phép chia sẻ.

– Nếu nhiều tiến trình sử dụng tài nguyên đồng thời, có nguy cơ xảy ra các kết quả không dự đoán được do hoạt động của các tiến trình trên tài nguyên ảnh hưởng lẫn nhau.

Để giải quyết vấn đề, cần bảo đảm tiến trình độc quyền truy xuất tài nguyên, nghĩa là hệ thống phải kiểm soát sao cho tại một thời điểm, chỉ có một tiến trình được quyền truy xuất một tài nguyên không thể chia sẻ.

4.3.2 Yêu cầu phối hợp (Synchronization)

Nhìn chung, mối tương quan về tốc độ thực hiện của hai tiến trình trong hệ thống là không thể biết trước, vì điều này phụ thuộc vào nhiều yếu tố động như tần suất xảy ra các ngắt của từng tiến trình, thời gian tiến trình được cấp phát bộ xử lý… Có thể nói rằng các tiến trình hoạt động không đồng bộ với nhau. Nhưng có những tình huống các tiến trình cần hợp tác trong việc hoàn thành tác vụ, khi đó cần phải đồng bộ hóa hoạt động của các tiến trình, ví dụ một tiến trình chỉ có thể xử lý nếu một tiến trình khác đã kết thúc một công việc nào đó …

4.3.3 Bài toán đồng bộ hoá

4.3.3.1 Vấn đề tranh đoạt điều khiển (race condition)

Giả sử có hai tiến trình P1 và P2 thực hiện công việc của các kế toán, và cùng chia sẻ một vùng nhớ chung lưu trữ biến taikhoan phản ánh thông tin về tài khoản. Mỗi tiến trình muốn rút một khoản tiền tienrut từ tài khoản:

```
if (taikhoan - tienrut >=0)
taikhoan = taikhoan - tienrut;
else
error(“khong the rut tien !”);
```

Giả sử trong tài khoản hiện còn 800, P1 muốn rút 500 và P2 muốn rút 400. Nếu xảy ra tình huống như sau:

- Sau khi đã kiểm tra điều kiện (taikhoan - tienrut >=0) và nhận kết quả là 300, P1 hết thời gian xử lý mà hệ thống cho phép, hệ điều hành cấp phát CPU cho P2.
- P2 kiểm tra cùng điều kiện trên, nhận được kết quả là 400 (do P1 vẫn chưa rút tiền) và rút 400. Giá trị của taikhoan được cập nhật lại là 400.
- Khi P1 được tái kích hoạt và tiếp tục xử lý, nó sẽ không kiểm tra lại điều kiện (taikhoan - tienrut >=0)-vì đã kiểm tra trong lượt xử lý trước- mà thực hiện rút tiền. Giá trị của taikhoan sẽ lại được cập nhật thành -100. Tình huống lỗi xảy ra. Các tình huống tương tự như thế - có thể xảy ra khi có nhiều hơn hai tiến trình đọc và ghi dữ liệu trên cùng một vùng nhớ chung, và kết quả phụ thuộc vào sự điều phối tiến trình của hệ thống- được gọi là các tình huống tranh đoạt điều khiển (race condition) .

4.3.3.2 Miền găng (critical section)

Để ngăn chặn các tình huống lỗi có thể nảy sinh khi các tiến trình truy xuất đồng thời một tài nguyên không thể chia sẻ, cần phải áp đặt một sự truy xuất độc quyền trên tài nguyên đó: khi một tiến trình đang sử dụng tài nguyên, thì những tiến trình khác không được truy xuất đến tài nguyên.

Đoạn chương trình trong đó có khả năng xảy ra các mâu thuẫn truy xuất trên tài nguyên chung được gọi là miền găng (critical section). Trong ví dụ trên, đoạn mã:

```
if (taikhoan - tienrut >=0)
taikhoan = taikhoan - tienrut;
```

của mỗi tiến trình tạo thành một miền găng.

Có thể giải quyết vấn đề mâu thuẫn truy xuất nếu có thể bảo đảm tại một thời điểm chỉ có duy nhất một tiến trình được xử lý lệnh trong miền găng. Một phương pháp giải

quyết tốt bài toán miền găng cần thõa mãn 4 điều kiện sau:

- Không có hai tiến trình cùng ở trong miền găng cùng lúc.
- Không có giả thiết nào đặt ra cho sự liên hệ về tốc độ của các tiến trình, cũng như về số lượng bộ xử lý trong hệ thống.
- Một tiến trình tạm dừng bên ngoài miền găng không được ngăn cản các tiến trình khác vào miền găng.
- Không có tiến trình nào phải chờ vô hạn để được vào miền găng.

Câu hỏi ôn tập

1. Thế nào là độc quyền truy xuất (mutual exclusion)? Cho ví dụ.
2. Race condition là gì? Nêu một ví dụ cụ thể.
3. Miền găng là gì? Một giải pháp giải quyết bài toán miền găng phải thỏa mãn những điều kiện nào?

Bài 10: Nhu cầu đồng bộ hóa — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: hai thread cùng tăng số lượt đăng ký. Phép đọc–cộng–ghi gồm nhiều bước; nếu cùng đọc 10 rồi đều ghi 11 thì mất một cập nhật. Đây là mô hình minh họa race condition, không phải bảo đảm mọi runtime đều xen kẽ đúng như vậy.

Tự luyện: viết các bước xen kẽ gây kết quả sai và đánh dấu vùng tới hạn. **Hướng giải:** tìm dữ liệu dùng chung và bất biến cần giữ, rồi chọn đồng bộ phù hợp. Deadlock khác race: ví dụ T1 giữ A chờ B, T2 giữ B chờ A nên cả hai không tiến được.

Nguồn học tiếp

https://pages.cs.wisc.edu/~remzi/OSTEP/